CO326 – Edge AI & Industrial IoT

Mini Project Report

Edge AI Motor Condition Monitoring System

Real-time Vibration Anomaly Detection using Isolation Forest

1. Introduction

This report documents the design and implementation of an Edge AI-based motor condition monitoring system. The system detects motor faults in real time by analyzing vibration data using machine learning, communicating results over MQTT, and presenting a live dashboard.

The system follows a complete Edge AI pipeline, a vibration simulator generates sensor readings, an Isolation Forest model classifies each reading as NORMAL or FAULT, and fault events trigger a simulated relay shutdown. All telemetry is published to a Mosquitto MQTT broker, with Node-RED subscribing to visualize the results.

1.1 Objectives

- Acquire motor vibration data (simulated in software using Python)
- Apply unsupervised machine learning (Isolation Forest) for anomaly detection
- Communicate sensor data and predictions over MQTT using Mosquitto broker
- Visualize live results via a Node-RED dashboard with charts, gauges, and alerts
- Containerize the full system using Docker and Docker Compose

1.2 Scope

The hardware layer is designed around an ESP32 microcontroller, MPU6050 accelerometer, relay module, and a physical motor. In this project, real hardware based data generation, acquisition, and processing are simulated rather than physically implemented. This approach allows the system to showcase a complete production level software architecture and AI-driven edge monitoring logic, enabling development and testing without the need for actual hardware components.

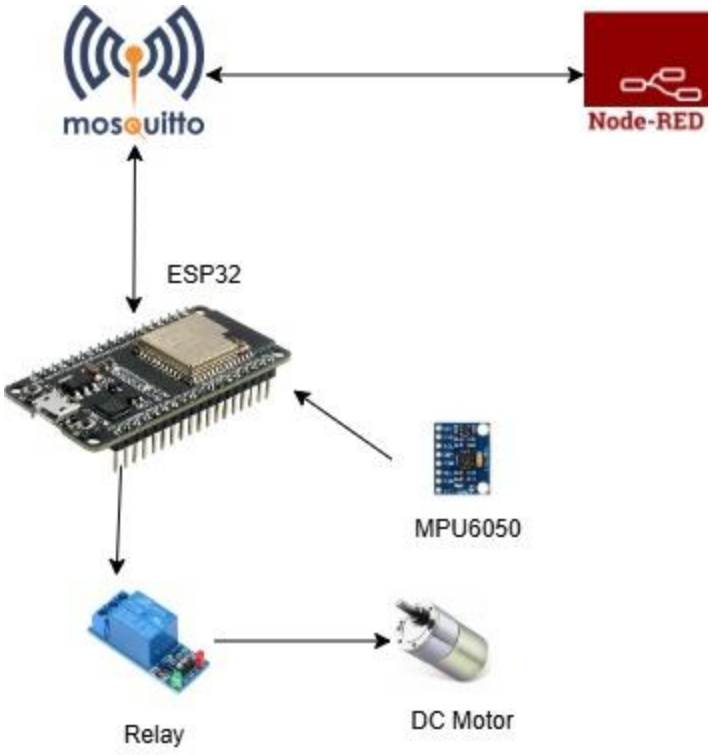


Figure 1: Hardware Setup

2. System Architecture

The system is organized into four major layers that mirror a real industrial IoT deployment:
Data acquisition, edge AI processing, MQTT communication, and visualization.

2.1 Architecture Overview

The end-to-end data flow is as follows

- A Python-based vibration simulator generates one vibration reading (in g) every second, mimicking a rotating motor
- A stateful anomaly detector maintains a sliding window of recent readings, computes derived features, and passes them to the trained Isolation Forest model.
- The model classifies each reading as NORMAL or FAULT. A detected fault sets the motor state to OFF, simulating relay-based shutdown.
- The MQTT client publishes telemetry, prediction labels, AI confidence scores, and fault alerts to a Mosquitto broker.
- Node-RED subscribes to the MQTT topics and renders a real time dashboard with charts, gauges, and status indicators.

2.2 Architecture Diagram

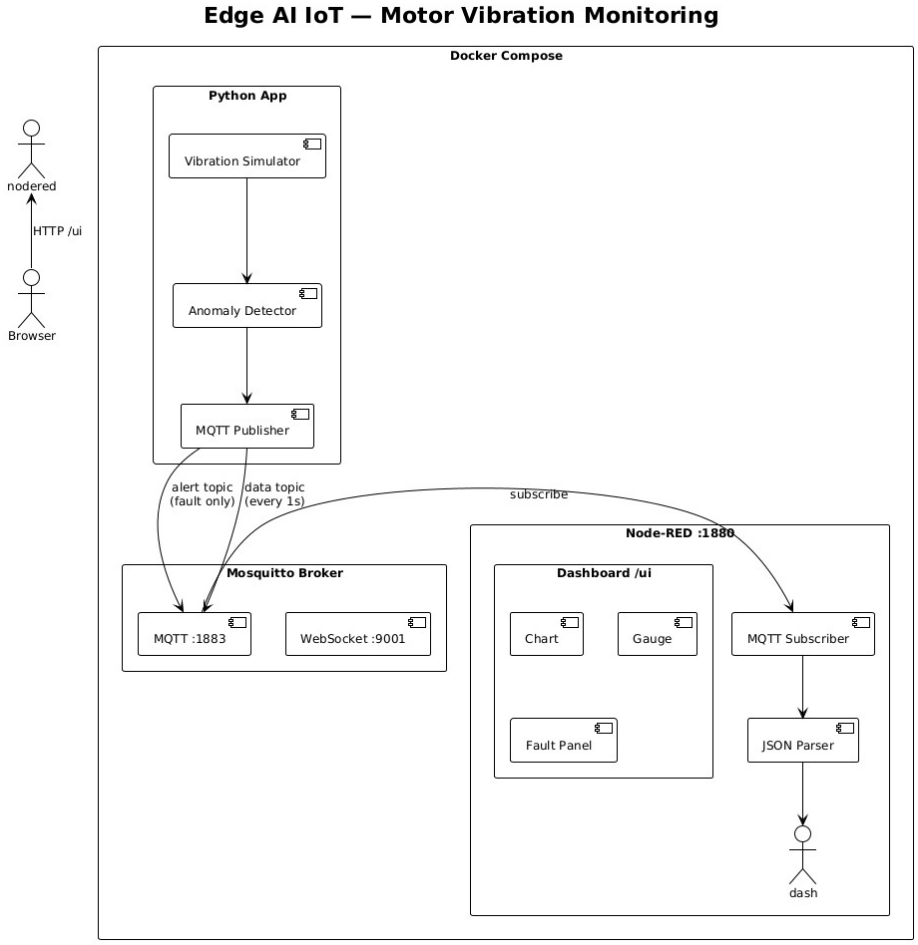


Figure 2: System Architecture Data Flow

2.3 Technology Stack

Component	Technology	Purpose
Edge AI Runtime	Python 3.11, scikit-learn	Model inference and feature extraction
Anomaly Model	Isolation Forest	Unsupervised fault classification
Feature Scaling	StandardScaler (scikit-learn)	Normalize features before inference
Data Simulation	NumPy, Python	Generate synthetic vibration readings
Message Broker	Mosquitto MQTT	Topic-based message routing
MQTT Client	paho-mqtt	Publish/subscribe to MQTT topics
Dashboard	Node-RED + node-red-dashboard	Real-time visualization
Containerization	Docker, Docker Compose	Service orchestration
Testing	pytest	Unit and integration tests

3. Task Completion

Task 1 & 2 - Environment Setup & GitHub Repository

The project repository was set up under the group's GitHub account with the required folder structure. Docker Compose was configured to orchestrate three containers: the Python AI service, the Mosquitto MQTT broker, and Node-RED. All services start with a single command

```
docker-compose up --build
```

The repository maintains the following structure

```
├── docker-compose.yml
├── requirements.txt
├── mosquitto/mosquitto.conf
├── node-red/   (flows.json, Dockerfile, settings.js)
├── python/     (main.py, anomaly_detector.py, mqtt_client.py,
│               vibration_simulator.py, Dockerfile)
└── docs/       (report)
```

Task 3 - Data Acquisition (Vibration Simulator)

Contributor: E/20/248

Motor Vibration Simulation Methodology

1. Objective

The purpose of this simulation is to generate realistic motor vibration data that mimics the output of an accelerometer such as the MPU6050. The simulated data is used for developing and evaluating the anomaly detection pipeline without requiring physical hardware.

2. Signal Modeling

Motor vibration is modeled as a **time-dependent signal** composed of periodic motion, harmonics, and noise. The general formulation is:

$$v(t) = \sum_{n=1}^{N} A_n \sin(2\pi n f t) + \mathcal{N}(0, \sigma) + F(t)$$

Where:

- $v(t)$is the vibration signal
- $f$is the fundamental frequency (related to motor speed)
- nfrepresents harmonic frequencies
- $A_n$are harmonic amplitudes
- $\mathcal{N}(0, \sigma)$is Gaussian noise
- $F(t)$represents fault induced disturbances

3. Normal Operating Condition

Under normal conditions, motor vibration is primarily periodic due to rotation

$$v_{normal}(t) = A_1 \sin(2\pi f t) + A_2 \sin(4\pi f t) + A_3 \sin(6\pi f t) + \mathcal{N}(0, \sigma)$$

This includes

- Fundamental frequency component
- Harmonics caused by mechanical structure
- Low-amplitude noise

4. Fault Modeling

Fault conditions are introduced using different disturbance patterns

4.1 Spike Fault

Sudden high-amplitude impulses:

$$v(t) = v(t) + S(t)$$

Where $S(t)$is a random high-value spike.

4.2 Burst Fault

Short-duration high-energy vibration:

$$v(t) = v(t) + B(t)$$

Where $B(t)$is a noisy signal over a short interval.

4.3 Imbalance / Modulation Fault

Amplitude modulation due to mechanical imbalance:

$$v_{fault}(t) = v_{normal}(t) \cdot (1 + \alpha \sin(2\pi f_m t))$$

Where

- $\alpha$is modulation depth
- $f_m$is modulation frequency

5. Feature Extraction

To support anomaly detection, statistical features are computed over a sliding window

- Mean
- Standard deviation
- Minimum and maximum
- Absolute difference between consecutive samples

These features capture both steady state behavior and transient anomalies.

6. Advantages of the Simulation

- Enables rapid development without hardware
- Allows controlled generation of both normal and fault data
- Supports reproducibility for training and testing
- Integrates seamlessly with the AI pipeline

7. Limitations

Despite its usefulness, the simulation has several limitations

1. **Simplified physics**
   Real motor dynamics involve complex mechanical interactions, which are not fully modeled.
2. **No frequency-domain realism**
   Effects such as resonance, damping, and structural vibration modes are not included.
3. **Synthetic fault patterns**
   Faults are artificially generated and may not perfectly represent real-world failures.
4. **No sensor imperfections**
   Real sensors introduce bias, drift, and calibration errors that are not simulated.
5. **Lack of environmental influence**
   External factors such as load variation, temperature, and mounting conditions are not considered.

Task 4 - MQTT Communication

Contributor: E/20/158

The MQTT client uses the paho-mqtt library to connect to the Mosquitto broker and publish messages on the following topics:

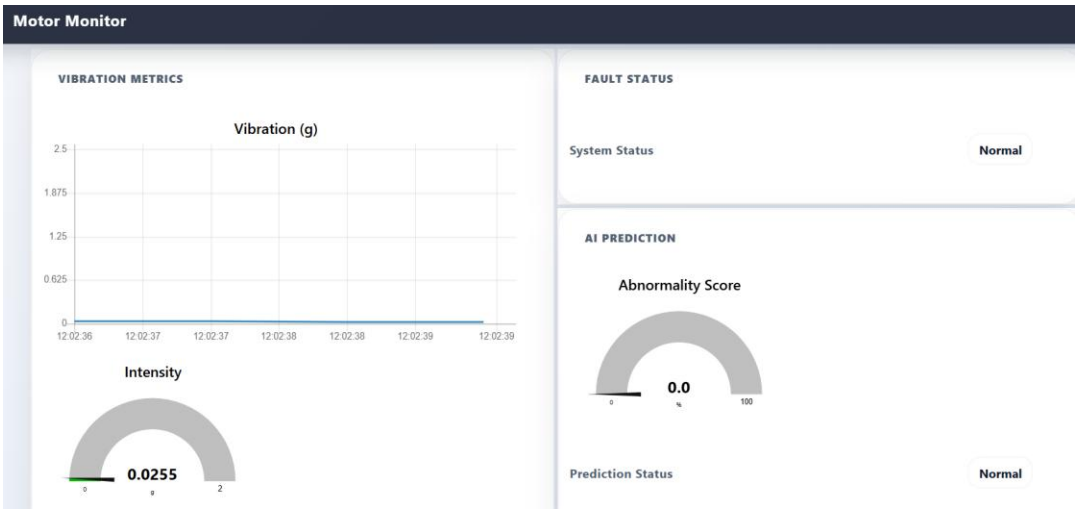
MQTT Topic	Description
sensors/group/motor/data	Raw vibration reading and derived features
sensors/group/motor/prediction	AI classification result (NORMAL / FAULT)
sensors/group/motor/confidence	Model confidence score
alerts/group/motor/status	Fault alert with motor ON/OFF state

Task 5 - Dashboard Visualization

Contributor: E/20/453

The Node-RED dashboard was built using node-red dashboard nodes and subscribes to all relevant MQTT topics. The dashboard includes

- Live vibration time-series chart updated every second
- Gauge showing current vibration magnitude in g
- AI prediction label display (NORMAL / FAULT) with color coding
- Confidence score indicator showing model certainty
- Motor/relay status panel showing ON or OFF state
- Fault alert panel with timestamp of last detected fault



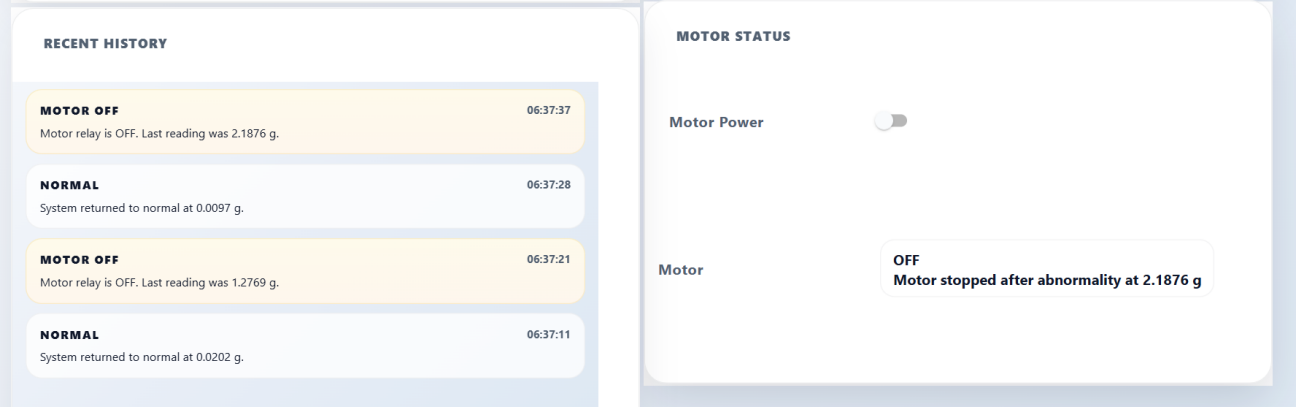


Figure 1:Dashboard UI

Task 6 - Edge AI Implementation (Anomaly Detector)

Contributor: E/20/300

The anomaly detection model is an Isolation Forest, an unsupervised algorithm well-suited for detecting rare anomalies without requiring labeled training data at inference time. Key design decisions:

- Contamination parameter set to 0.15, matching the 15% fault rate in the synthetic dataset.
- Features are normalized using StandardScaler before model input.
- A sliding window of recent readings maintains rolling statistics (mean, std, min, max).
- The feature vector passed to the model includes: current vibration, rolling mean, rolling std, rolling min, rolling max, and delta from the previous reading.

Training is performed offline from the synthetic dataset and the model is saved with joblib. At runtime, only inference is performed the model is not retrained.

Task 7 - Alert Integration

When the Isolation Forest classifies a reading as a fault, the system immediately

- Sets the motor state to OFF (simulating relay de-energization)
- Publishes a fault alert to the alerts MQTT topic
- Displays the alert prominently in the Node-RED dashboard

Task 8 & 9 - Testing, Debugging & Final Demo

The system was tested end-to-end using pytest for unit tests on the Python components. Integration testing verified the full pipeline: simulator output → feature extraction → model inference → MQTT publish → Node-RED display.

4. AI Model Details

4.1 Model Selection

The Isolation Forest algorithm was chosen for the following reasons:

- Unsupervised: does not require labeled fault data at training time, only a representative normal dataset.
- Computationally efficient: suitable for edge deployment with low inference latency.
- Robust to high-dimensional feature spaces and outlier contamination.

4.2 Dataset Generation

A synthetic labeled dataset was generated with the following characteristics:

Class	Proportion	Generation Method
Normal	~85%	Low-amplitude sine wave + Gaussian noise, clamped to ≥ 0
Fault (spike)	~8%	High-amplitude single impulse
Fault (burst)	~7%	Sustained high-energy irregular random values

4.3 Feature Engineering

Each sample saved to the dataset and fed to the model includes the following features:

Feature	Description
vibration	Current raw vibration reading (g)
rolling_mean	Mean of the last N readings
rolling_std	Standard deviation of the last N readings
rolling_min	Minimum of the last N readings
rolling_max	Maximum of the last N readings
delta	Absolute change from the previous reading
label	Ground-truth class: 0 = normal, 1 = fault (training only)

4.4 Training Pipeline

- Load synthetic labeled dataset from CSV
- Split into training and test sets (80/20)
- Standardize features with StandardScaler
- Train Isolation Forest with contamination=0.15 on training set
- Run inference on test set and map Isolation Forest output to normal/fault labels
- Evaluate using classification report and confusion matrix
- Save model and scaler to disk with joblib for runtime use

5. Team Contributions

Student ID	Module(s)	Responsibilities
E/20/453	Node-RED	Designed and implemented the Node-RED dashboard; configured all MQTT subscriptions; created UI flows for chart, gauge, confidence, fault alert, and motor status panels.
E/20/300	Anomaly Detector + Mosquitto	Implemented the Isolation Forest anomaly detection model; designed the feature extraction pipeline with rolling statistics; configured the Mosquitto MQTT broker.
E/20/158	MQTT Client + Dockerfile	Implemented the paho-mqtt client for publishing telemetry and alerts; wrote the Python service Dockerfile for containerized deployment.
E/20/248	Vibration Simulator + docker-compose	Implemented the synthetic vibration generator for normal and fault scenarios; configured docker-compose.yml to orchestrate all services

6. Challenges & Solutions

Challenge	Solution
No physical hardware available	Implemented a realistic Python simulator covering normal and two fault modes, preserving the pipeline integrity.
Isolation Forest outputs scores, not labels	Mapped Isolation Forest -1/1 output to fault/normal binary labels using a consistent threshold.
Rolling features not stable at startup	Withheld data from the dataset until the rolling window was fully populated, ensuring feature stability from record 1.
Docker networking between services	Used Docker Compose service names as MQTT hostnames (e.g., mosquitto) to enable inter-container communication.
Node-RED dashboard latency	Tuned the MQTT QoS and Node-RED rate-limiter settings to achieve smooth 1 second chart updates.

7. Results & Evaluation

The system successfully demonstrated the complete Edge AI pipeline:

- The vibration simulator produced stable, realistic readings at 1 Hz over extended periods.
- The Isolation Forest correctly identified fault conditions with low false-positive rates, consistent with the 15% contamination tuning.
- MQTT telemetry was delivered reliably, with Node-RED dashboard updates visible within approximately one second of each reading.
- Simulated relay shutdown (motor OFF state) triggered correctly on every detected fault event.
- The full system launched successfully from a single docker-compose up --build command.

8. Future Improvements

- Compute vibration magnitude from raw MPU6050 axis data ($\sqrt{(ax^2 + ay^2 + az^2)}$) rather than a scalar simulator.
- Add online learning to adapt the model to motor aging and changing operating conditions.
- Implement email or SMS fault notifications via Node-RED alongside the dashboard alert.
- Extend the dashboard with historical fault logging and trend analysis.

9. Conclusion

This project successfully demonstrated a complete Edge AI motor monitoring pipeline, from synthetic sensor data generation through machine learning inference to real time dashboard visualization. The Isolation Forest model, combined with rolling statistical features, provides reliable anomaly detection without requiring labeled fault data at runtime. The containerized architecture using Docker Compose ensures easy deployment and reproducibility.

The project meets all required deliverables and provides a solid foundation for extension to live hardware.